

Módulo: Patrón de Comportamiento OBSERVER (Observador)

1. Introducción y Definición

¿Qué es? El patrón Observer define una dependencia uno-a-muchos entre objetos, de tal manera que cuando un objeto (el **Sujeto** o *Subject*) cambia su estado, todos sus dependientes (los **Observadores** o *Observers*) son notificados y actualizados automáticamente.

Categoría: Patrón de Comportamiento (Behavioral). **Objetivo Principal:** Desacoplar el objeto que genera el evento (Sujeto) de los objetos que reaccionan a él (Observadores). Promueve el bajo acoplamiento (*loose coupling*).

Cita Clave (GoF): "Define una dependencia uno-a-muchos entre objetos para que cuando un objeto cambie de estado, todos sus dependientes sean notificados y actualizados automáticamente." — Gamma, Helm, Johnson, Vlissides (1994).

2. El Problema que Resuelve

Sin este patrón, los desarrolladores suelen caer en el **acoplamiento rígido**:

- El objeto emisor necesita conocer explícitamente a los receptores para llamar a sus métodos.
- Si añadimos un nuevo receptor, debemos modificar el código del emisor (violando el principio **Open/Closed**).
- Es difícil reutilizar el emisor en otro contexto porque está "atado" a clases específicas.

Solución Observer: El Sujeto no sabe *quiénes* son sus observadores, solo sabe que implementan una interfaz común (`update()`). Esto permite añadir o quitar observadores en tiempo de ejecución sin tocar el código del Sujeto.

3. Analogías para una Comprensión Intuitiva

Analogía 1: La Suscripción a un Canal de YouTube / Newsletter

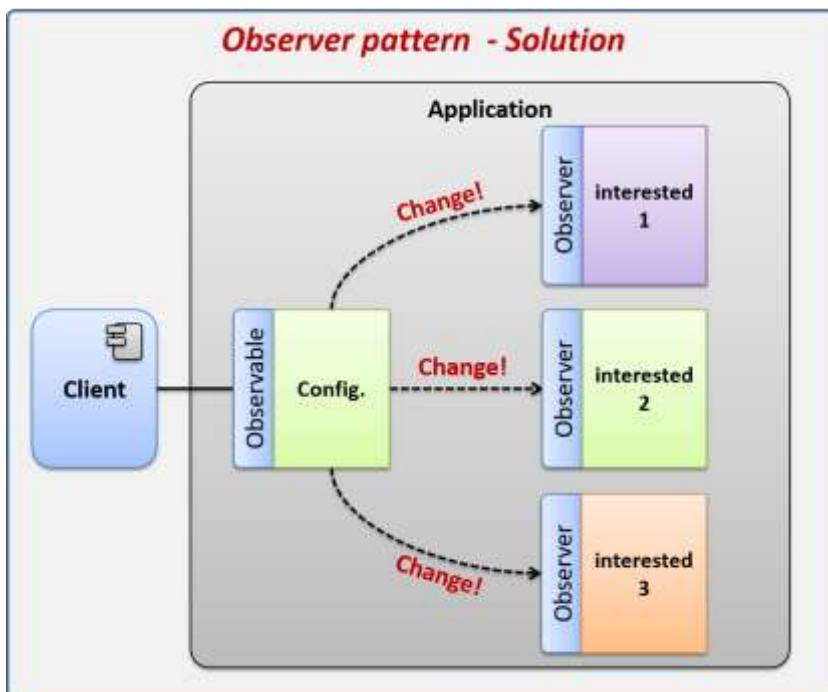
- **Sujeto (Subject):** El YouTuber o la revista.
- **Observadores (Observers):** Los suscriptores.
- **Mecánica:**

1. Te **suscribes** (attach/subscribe) al canal.
2. Cuando el YouTuber sube un video nuevo (cambio de estado), YouTube notifica a *todos* los suscriptores.
3. Tú decides qué hacer con esa notificación (verlo, ignorarlo, compartirlo).
4. Si te **desuscribes** (detach/unsubscribe), dejas de recibir notificaciones. El YouTuber no deja de subir videos, simplemente tú ya no estás en la lista.

Analogía 2: El Sistema de Alerta de Tsunami

- **Sujeto:** El sismógrafo central que detecta movimiento sísmico.
 - **Observadores:** Las sirenas de la costa, las apps móviles de emergencia, los sistemas de cierre automático de compuertas.
 - **Mecánica:** Cuando el sismógrafo detecta un terremoto fuerte, notifica a todos los sistemas conectados. Cada sistema reacciona diferente (la sirena suena, la app envía push, la compuerta se cierra), pero el sismógrafo no necesita saber cómo funciona cada uno, solo que deben ser alertados.
-

4. Estructura Técnica (Diagrama Lógico)



El patrón consta de cuatro roles principales:

1. Subject (Sujeto):

- Mantiene una lista de Observadores.
- Provee métodos para `attach()`, `detach()` y `notify()`.
- Contiene el estado que interesa a los observadores.

2. Observer (Observador - Interfaz):

- Define una interfaz común (ej. `update()`) para todos los observadores.

3. ConcreteSubject (Sujeto Concreto):

- Almacena el estado específico.
- Envía notificaciones a sus observadores cuando su estado cambia.

4. ConcreteObserver (Observador Concreto):

- Implementa la interfaz Observer.
- Mantiene una referencia al ConcreteSubject (opcional, para obtener detalles).
- Reacciona a la notificación ejecutando su lógica específica.

5. Aplicación Práctica según Contexto

A. En Back-End (Server-Side)

Uso Crítico: Arquitecturas orientadas a eventos y desacoplamiento de lógica de negocio.

- **Ejemplo Real:** Sistema de Pedidos E-commerce.
 - **Evento:** Un pedido cambia de estado a "ENVIADO".
 - **Sujeto:** OrderService.
 - **Observadores:**
 1. EmailService: Envía confirmación al cliente.
 2. InventoryService: Actualiza el stock disponible.
 3. AnalyticsService: Registra el evento para dashboards.

- **Beneficio:** Si mañana queremos añadir un observador SMSservice, solo lo creamos y lo subscribimos. No tocamos ni una línea de código en OrderService.
- **Tecnologías:** Implementación nativa en Java (java.util.Observer -deprecated pero conceptualmente válido), Spring Events (@EventListener), o brokers de mensajes como Kafka/RabbitMQ (versión distribuida del patrón).

B. En Front-End (Client-Side)

Uso Crítico: Reactividad y gestión de estado.

- **Ejemplo Real:** Frameworks Modernos (React, Vue, Angular).
 - **Sujeto:** El Store global (Redux, Vuex, Pinia) o un objeto reactivo (Vue 3 ref/reactive).
 - **Observadores:** Los componentes de la UI.
 - **Mecánica:**
 1. El componente se "suscribe" al estado.
 2. Cuando el usuario hace clic y cambia el estado (ej. user.name = "Montero"), el Store notifica a los componentes.
 3. Los componentes afectados se re-renderizan automáticamente.
 - **Nota Técnica:** Vue 3 usa el patrón **Proxy** internamente para detectar estos cambios y actuar como mecanismo de notificación eficiente.

6. Comparativa con Otros Patrones (Para evitar confusiones)

Patrón	Diferencia Clave con Observer
Publisher-Subscriber (Pub/Sub)	Son similares, pero Pub/Sub suele ser asíncrono y usar un intermediario (Broker/Bus) para desacoplar totalmente a Sujetos y Observadores (no se conocen ni siquiera por interfaz). Observer es usualmente síncrono y directo.
Mediator	El Mediator centraliza la comunicación compleja entre muchos objetos. Observer es una relación uno-a-muchos más simple y direccional (Sujeto -> Observadores).

Patrón

Diferencia Clave con Observer

Decorator

Decorator añade responsabilidad *estructuralmente* envolviendo un objeto. Observer añade comportamiento *reactivo* ante cambios de estado.

7. Ejemplo de Código (Conceptual en TypeScript/Java)

```
1 // 1. Interfaz Observer
2 interface Observer {
3     update(message: string): void;
4 }
5
6 // 2. Sujeto Concreto
7 class NewsPublisher {
8     private observers: Observer[] = [];
9     private latestNews: string = "";
10
11     // Suscribirse
12     subscribe(observer: Observer): void {
13         this.observers.push(observer);
14     }
15
16     // Desuscribirse
17     unsubscribe(observer: Observer): void {
18         this.observers = this.observers.filter(obs => obs !== observer);
19     }
20
21     // Cambiar estado y notificar
22     setNews(news: string): void {
23         this.latestNews = news;
24         console.log(`Publisher: Nueva noticia publicada: ${news}`);
25         this.notifyObservers();
26     }
27
28     private notifyObservers(): void {
29         for (const observer of this.observers) {
30             observer.update(this.latestNews);
31         }
32     }
33 }
34
35 // 3. Observadores Concretos
36 class MobileApp implements Observer {
37     update(message: string): void {
38         console.log(`[Mobile App] Notificación Push recibida: ${message}`);
39     }
40 }
41
```

```

42 class WebDashboard implements Observer {
43     update(message: string): void {
44         console.log(`[Web Dashboard] Actualizando feed en tiempo real: ${message}`);
45     }
46 }
47
48 // 4. Cliente (Uso)
49 const publisher = new NewsPublisher();
50 const mobile = new MobileApp();
51 const web = new WebDashboard();
52
53 publisher.subscribe(mobile);
54 publisher.subscribe(web);
55
56 publisher.setNews("¡Nuevo curso de Patrones de Diseño disponible!");
57 // Salida:
58 // Publisher: Nueva noticia publicada: ¡Nuevo curso...!
59 // [Mobile App] Notificación Push recibida: ¡Nuevo curso...!
60 // [Web Dashboard] Actualizando feed en tiempo real: ¡Nuevo curso...!

```

8. Ventajas y Desventajas

Ventajas:

- **Principio Open/Closed:** Puedes introducir nuevas clases de observadores sin cambiar el código del sujeto.
- **Desacoplamiento:** El Sujeto no depende de clases concretas de Observadores.
- **Comunicación dinámica:** Las relaciones se establecen en tiempo de ejecución.

Desventajas:

- **Orden de notificación indefinido:** Si un observador depende de otro, el orden de actualización puede causar errores si no se gestiona bien.
 - **Fugas de Memoria (Memory Leaks):** En Frontend, si no se desuscriben los observadores (ej. al destruir un componente React/Vue), pueden permanecer en memoria consumiendo recursos.
 - **Complejidad de Depuración:** Puede ser difícil rastrear quién notificó a quién en sistemas grandes con muchas suscripciones.
-

9. Referencias Bibliográficas

1. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Capítulo 5: Behavioral Patterns, sección Observer).
 2. **Freeman, E., & Robson, E.** (2020). *Head First Design Patterns* (2nd ed.). O'Reilly Media. (Capítulo 2: The Observer Pattern - El ejemplo del clima es clásico y didáctico).
 3. **Material del Curso:**
 - *PRESENTACION S13.pdf*: Sección "Patrones de Comportamiento", apartado "Observer: Fundamental para arquitecturas orientadas a eventos".
 - *PRESENTACION Semana 9.pdf*: Clasificación general de patrones GoF y comparación estructural.
 4. **Refactoring.Guru.** (n.d.). *Observer Pattern*. Recuperado de <https://refactoring.guru/design-patterns/observer>
-

Actividad Sugerida para Estudiantes

Reto de Diseño: Diseñar un sistema de monitoreo de servidores.

- **Sujeto:** ServerMonitor (detecta CPU > 90%).
- **Observadores:**
 1. LogWriter: Guarda el evento en un archivo.
 2. AdminAlert: Envía un email al administrador.
 3. AutoScaler: Añade más instancias al servidor (acción correctiva).
- **Pregunta:** ¿Qué pasaría si el AutoScaler falla? ¿Cómo garantiza el patrón Observer que el LogWriter siga funcionando? (Respuesta: El desacoplamiento asegura que el fallo de un observador no necesariamente detenga la notificación a los demás, si se manejan las excepciones correctamente en el bucle de notificación).